

Normalization is the process of organizing tables and attributes into a relational database in order to minimize data duplication, and eliminate anomalies such as insertion/deletion errors. The reason we decided it was important to perform database normalization is to make sure that our tables are easy to work with, update, and to maintain.

First, we made sure to follow the 1NF, which required each cell to hold one specific value (no lists, repeating groups, etc), simply known as atomisation. In order to follow this rule we had to split cells which contained multiple values into new entities. For example, in the unnormalized BAD_SUPERMARKET_DATA table there are violations such as waste_product_ids and inventory_product_ids. These columns contain multiple values which are separated by commas in one field.

Let's take an example extract (such that not all the attributes are included) of the FOOD_WASTE table:

BEFORE:

waste_id	supermarket_id	waste_product_ids	disposal_method_id
401	3	103,104,105	300
402	4	106	301

AFTER:

waste_id	supermarket_id	product_id	disposal_method_id
401	3	103	300
401	3	104	300
401	3	105	300
402	4	106	301

Secondly, we followed the 2NF rule. 2NF requires us to satisfy the 1NF rule and the database cannot have any partial dependencies. This means that all non-key attributes must be fully functionally dependent on the entire primary key. In our initial design we considered waste_id, supermarket_id and product_id as a composite primary key. Most of our non-key attributes did not directly depend on the composite primary key of waste_id + supermarket_id + product_id. More specifically attributes have a partial dependency on the composite key: quantity_wasted and waste_date depend on the waste_id; name and country only depend on supermarket_id; product_name, price and shelf_life only depend on product_id. To resolve this, we separated the data into different entities. waste_id became the primary key whilst supermarket_id and product_id became FK's linking to the SUPERMARKET and PRODUCT tables respectively. Each corresponding data was moved to their respective tables. For example, the name 'SUPERMARKET B' is no longer needed to be repeated for every waste and product instance associated with that supermarket.

The tables below show an example (it must however be noted that this was before the addition of the Foreign Keys of category_id, and supplier_id):

BEFORE:

waste_id	supermarket_id	product_id	quantity_wasted	waste_date	name	country	product_name	price	shelf_life
401	3	1896	5	01/01/28	Supermarket A	GR	apricot	5	3
402	4	1753	15	01/12/26	Supermarket B	NL	melon	10	3
403	4	5842	5	01/02/29	Supermarket B	NL	bread	7	7

AFTER:

FOOD_WASTE

waste_id	supermarket_id	product_id	quantity_wasted	waste_date
401	3	1896	5	01/01/28
402	4	1753	15	01/12/26
403	4	5842	5	01/02/29

SUPERMARKET

supermarket_id	name	country
3	Supermarket A	GR
4	Supermarket B	NL

PRODUCT

product_id	product_name	price	shelf_life
1896	apricot	5	3
1753	melon	10	3
5842	bread	7	7

Lastly, we followed the 3NF rule. This rule states that a table must satisfy the 2NF and have no transitive dependencies. Thus, meaning that non-key attributes should depend directly on the primary key and not other non-key attributes. In our table structure, even after separating the PRODUCT and FOOD_WASTE, several non key attributes depended on other non key attributes. For example, supplier 'name' and phone_number depend on the supplier_id and category_name depend on category_id rather than directly on the product_id. If a supplier updated their phone number we would suffer an anomaly, forcing us to update every single row associated with that supplier. To fix this we stripped out these non key text string and replaced them with relationships. For example our main extractions happened towards category, supplier and disposal_method.

BEFORE:

Product_id	name	price	category_id	category_name	supplier_id	supplier_name	phone_number
101	apple	0.99	10	fresh produce	21	global G	3456778
102	bananas	1.2	10	fresh produce	21	global G	3456778

AFTER:

PRODUCT

Product_id PK	name	price	shelf_life	category_id FK	supplier_id FK
101	apple	0.99	14	10	21
102	bananas	1.2	7	10	21

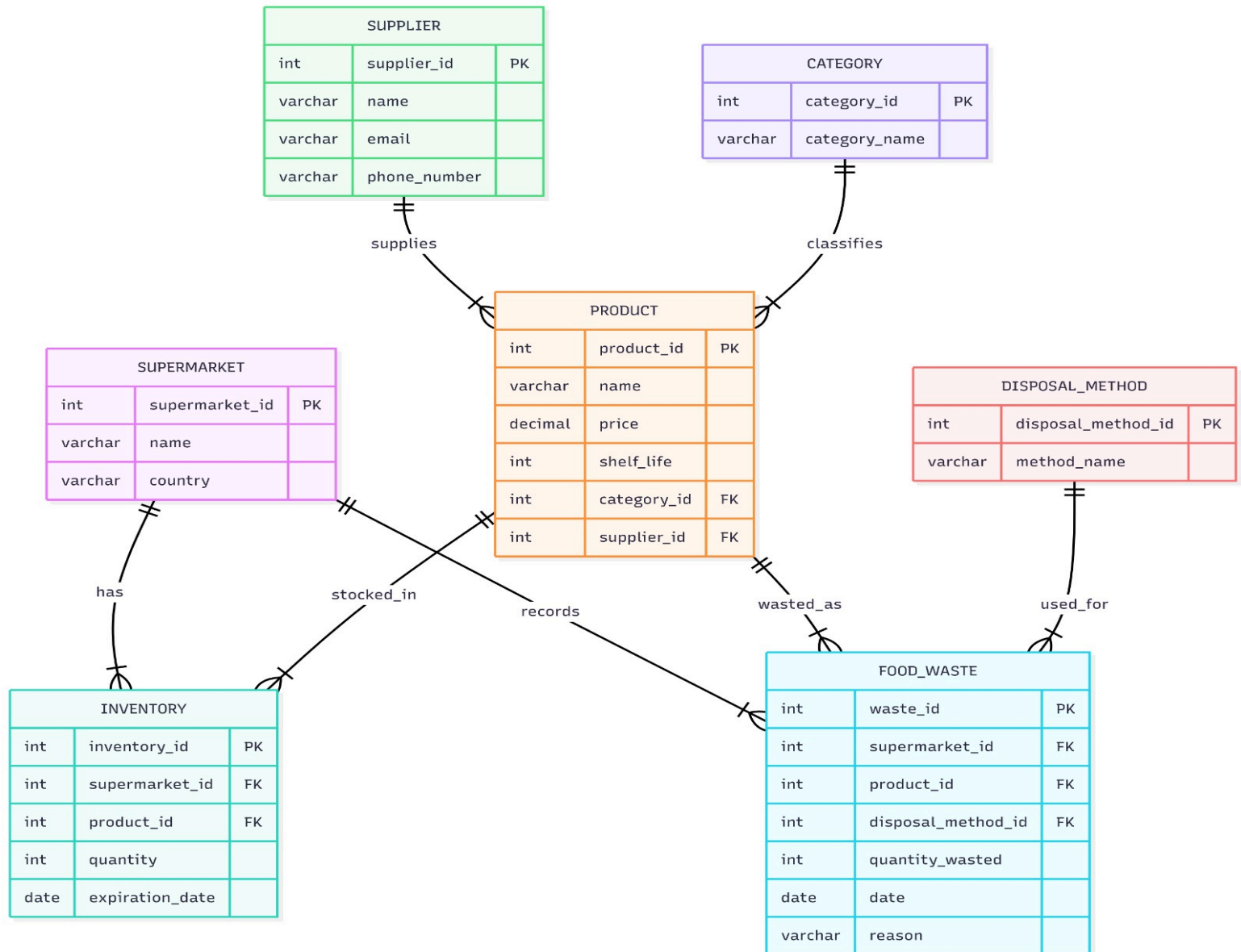
SUPPLIER

supplier_id PK	name	email	phone_number
21	global G	globalG.@gmail.com	3456778
22	global K	globalK@gmail.com	3452077

CATEGORY

category_id PK	category_name
10	fresh produce
18	frozen

Figure 1: Supermarket ERD diagram.



BAD_SUPERMARKET_DATA		
int	supermarket_id	PK
int	product_id	PK
int	waste_id	PK
varchar	supermarket_name	
varchar	country	
varchar	product_name	
decimal	price	
int	shelf_life	
int	category_id	
varchar	category_name	
int	supplier_id	
varchar	supplier_name	
varchar	supplier_email	
varchar	supplier_phone_number	
int	inventory_id	
varchar	inventory_product_ids	
int	quantity	
date	expiration_date	
varchar	waste_supermarket_ids	
varchar	waste_product_ids	
int	disposal_method_id	
varchar	disposal_method_name	
int	quantity_wasted	
date	waste_date	

Figure 2: Scratch version ERD diagram.